

AgroSearch — Motor de Busca Inteligente

UNIPÊ — Centro Universitário de João Pessoa · Tópicos Avançados · Laboratório Prático 04
Prof. Me. Ricardo Roberto de Lima · Equipe: [Integrante 1] · [Integrante 2] · [Integrante 3]

1. Objetivo

Protótipo de motor de busca textual para os manuais técnicos da AgroTech Solutions. O técnico digita uma consulta e recebe os trechos mais relevantes, ranqueados por TF-IDF. Seguindo a restrição do desafio, **nada foi feito com scikit-learn**: pré-processamento, índice invertido, TF-IDF e cosseno foram implementados do zero em Python puro (*re*, *unicodedata*, *math*, *collections*), com interface em Streamlit.

2. Arquitetura (agrosearch_app.py)

- **Fase 1 — Pré-processamento**: *tokenizar* (*regex \w+*) → *normalizar* (minúsculas e remoção de acentos via NFD) → *remover_stopwords* (lista própria em português) → *stem* (stemmer por regras que remove o sufixo mais longo, preservando um radical de 3+ letras). Stopwords e stemming têm checkboxes na sidebar. Com as duas etapas ligadas o vocabulário cai de 47 para 33 termos.
- **Fase 2 — Índice invertido**: dicionário termo → lista de documentos, exibido em tabela (com df) e em JSON.
- **Fase 3 — TF-IDF**: $TF(t,d) = f(t,d)/|d|$; $IDF(t) = \log_{10}(N/df(t))$; $score(d)$ = soma do TF-IDF dos termos da consulta. A tabela é ordenada do maior para o menor, o vencedor fica destacado em verde, e um expander mostra o cálculo termo a termo.
- **Bônus — Cosseno**: vetores TF-IDF da consulta e dos documentos sobre o vocabulário inteiro e $\cos = q \cdot d / (|q||d|)$, lado a lado com o TF-IDF acumulado.

3. Exemplo: consulta "lagartas na soja"

Após o pipeline: [lagart, soj]. $df(lagart) = 2 \rightarrow IDF = \log_{10}(5/2) = 0,398$; $df(soj) = 3 \rightarrow IDF = \log_{10}(5/3) = 0,222$.

Doc	Termo	f/ d	TF	df	IDF	TF-IDF
Doc 1	soj	1/8	0.125	3	0.222	0.0277
Doc 2	lagart	1/7	0.143	2	0.398	0.0568
Doc 2	soj	1/7	0.143	3	0.222	0.0317
Doc 4	lagart	1/8	0.125	2	0.398	0.0497
Doc 4	soj	1/8	0.125	3	0.222	0.0277

Pos.	Documento	TF-IDF acumulado
1	Doc 2	0.0885
2	Doc 4	0.0775
3	Doc 1	0.0277
4	Doc 3	0.0000
5	Doc 5	0.0000

O Doc 2 vence o Doc 4 porque é mais curto (7 × 8 tokens): o mesmo par de termos pesa mais no TF. O Doc 1 só tem "soja", o termo menos discriminativo.

4. Observações

- Somar TF-IDF favorece documentos com **um** termo raro. No bônus, o cosseno considera a consulta inteira e penaliza documentos que tratam de muitos outros assuntos.
- O stemmer por regras junta "lagartas"/"lagarta" e "irrigação"/"irrigar", mas também produz radicais sem significado ("requ", "algod"). É o custo esperado de um stemmer heurístico, citado em aula.
- Termos da consulta fora do vocabulário recebem IDF 0 e aparecem num aviso na interface.

5. Divisão de tarefas

Integrante	Responsabilidade
[Integrante 1]	Pipeline de pré-processamento e stemmer (Fase 1)
[Integrante 2]	Índice invertido e cálculo de TF, IDF e TF-IDF (Fases 2 e 3)
[Integrante 3]	Interface Streamlit, bônus do cosseno e relatório

Execução: `pip install streamlit pandas` e `streamlit run agrosearch_app.py`.